

OSINT Report:

Objective:

The objective of this task was to understand passive reconnaissance and Open-Source Intelligence (OSINT): collect publicly available information about a practice domain using Python.

Practice Domain: example.com

It was used as the practice domain because it is a reserved domain intended for documentation, demonstrations, and testing purposes.

Tools and Libraries Used:

- Python-whois: to retrieve publicly available domain information
- Socket: to resolve the domain name to an IP Address
- Requests: to communicate with the IP geolocation API
- Ip-api.com: to obtain approximate location information for the resolved IP Address

Results: The script produced the following results:

```
Registrar: RESERVED-Internet Assigned Numbers Authority  
IP: 172.66.147.243  
Location: Toronto, Canada
```

Security Perspective:

OSINT can help security professionals understand what information is publicly exposed about a domain. Attackers may also use such information during reconnaissance, so organizations should limit unnecessary public exposure.

Ethical Considerations:

This task was performed only on the practice domain example.com using publicly available information. No exploitation, active scanning, or unauthorized access was performed.

Conclusion:

The task demonstrated how Python can automate simple passive reconnaissance using WHOIS, DNS resolution and IP geolocation.

Email Harvesting & Social Engineering Prep:

Objective:

The objective of this task was to understand how email harvesting works and how publicly available email addresses can be collected from a webpage using Python.

Lab Setup:

A local practice webpage was created and hosted on the local machine specifically for testing the email harvesting script. The webpage was prepared with fictional email addresses so the script could be tested safely in an authorized lab environment.

Tools Used: Python, requests, re and Python's built-in local HTTP server.

Method:

The Python script downloaded the HTML content of the local practice webpage using the requests library. A regular expression was then used to identify strings matching the format of an email address. The results were stored in a set to remove duplicates and then printed in the terminal.

Emails Found:

```
admin@example.com  
support@example.com  
security@example.com
```

How Attackers Could Use Harvested Emails:

Attackers can use publicly exposed email addresses to prepare for social engineering attacks such as phishing, spear phishing or impersonation. For example, knowing that an address belongs to a support or security role can help an attacker create a more convincing and targeted message. Harvested addresses may also be used to identify employees or organizational roles before attempting further SE.

Ethical Consideration:

This exercise was performed only on a local lab environment using fictional email addresses. No real users, companies or unauthorized systems were targeted.

Conclusion:

This task demonstrated how simple Python tools can collect exposed email addresses from webpage content and why organizations should be careful about publishing unnecessary contact information online.

Phishing Page Anatomy & Detection:

Objective:

The objective of this task was to understand common phishing indicators and build a Python-based URL phishing scorer. The script analyzes URL characteristics and assigns a risk score based on predefined suspicious patterns.

Tool Used: Python, urllib.parse and the re module

Method:

The script uses urlparse() to extract the hostname from each URL and then checks for several suspicious indicators. A score is increased when a URL matches one or more of the following rules:

- **No HTTPS -> +30 points:** If the URL does not start with https, the script adds 30 points. HTTPS encrypts communication between the browser and the website. The absence of HTTPS can be considered a warning sign, although it does not automatically mean that a website is malicious.
- **Suspicious keywords in the hostname -> +20 points for each keyword:** The script checks whether the hostname contains such as login, verify, secure, update, account, bank or paypal. These words are often used in phishing URLs to make a fake page appear legitimate or urgent. Each matching keyword adds 20 points.
- **Too Many Subdomains -> +25 points:** If the hostname contains more than three dots, the script adds 25 points. Phishing URLs may use many subdomains to make a malicious address look trustworthy. For example, a long hostname can make the beginning of the URL appear related to a legitimate service even though the actual registered domain is different.
- **IP Address Instead of a Domain Name -> +40 points:** If the hostname contains a numerical IP address instead of a normal domain name, the script adds 40 points. Legitimate public websites usually use recognizable domain names, while suspicious links may sometimes use direct IP addresses to hide the real destination.
- **Maximum Score -> 100 points**

Sample Results:

```
https://paypal-login.evil.com/verify -> Risk: 40%
https://github.com -> Risk: 0%
https://example.com -> Risk: 0%
http://example.com -> Risk: 30%
http://192.168.1.10/login -> Risk: 70%
https://secure-account.example.com -> Risk: 40%
https://update.example.com -> Risk: 20%
https://a.b.c.d.e.example.com -> Risk: 25%
https://bank-login.example.com -> Risk: 40%
https://docs.python.org -> Risk: 0%
```

Analysis:

URLs using HTTPS and containing no suspicious indicators received a score of 0%. URLs using HTTP received an additional 30 points. Hostnames containing words such as login, bank, secure, account or update received additional points. URLs with many subdomains or an IP address instead of a standard domain name also received higher scores.

Limitations:

The script is a simple rule-based detector. A low score does not guarantee that a URL is safe and a high score does not prove that a URL is malicious. The result only indicates that certain suspicious patterns were detected.

Ethical Considerations:

The test URLs were synthetic, reserved, or documentation-based examples, and the script only analyzed URL strings locally. No network requests, scanning, or interaction with external systems were performed.

Conclusion:

This task demonstrated how common phishing indicators can be detected automatically using Python. It also showed that URL-based detection can be useful as an initial warning mechanism, although additional security checks are required for reliable phishing detection.

Vishing and Smishing Simulation Scripts

Objective:

The objective of this task was to understand how vishing and smishing attacks use social engineering techniques to influence people. Three awareness-Training scenarios were created for IT support, banking and government impersonation.

Tool Used: Python standard library

Method:

A python function was created to generate structured vishing awareness scripts. Each script includes an attacker role, target organization, pretext, opener, hook and defensive red flags. The three scenarios were designed only for security-awareness training.

- Scenario1
 - IT support: the caller pretends to be from IT support and claims that there is a problem with the user's account. The caller then asks the user to confirm sensitive information.
 - Psychological Triggers: Authority is used because the caller claims to be part of the IT department. Fear is also used by suggesting that there is an account problem. This pressures the user to trust the caller and respond quickly.
- Scenario2
 - Bank representative: the caller pretends to be from a bank fraud department and reports a suspicious transaction. The caller then asks the user to provide a verification code.
 - Psychological Triggers: Fear is created by mentioning suspicious financial activity. Urgency is used to make the user feel that immediate action is necessary. Authority is also present because the caller claims to present the bank.
- Scenario3
 - Government officer: the caller pretends to be from a public-service office and asks the user to verify personal information. The caller also states that the process must be completed within the next 12 hours.
 - Psychological Triggers: Authority is used because the caller claims to represent a government organization. Urgency is created by the given 12-hour deadline. Pressure and fear are added by suggesting that delays or problems may occur if the user does not respond.

Red Flags and Defensive Actions:

Users should never share passwords, PINs, one-time verification codes, or sensitive personal information during an unexpected phone call. Suspicious calls should be ended, and the organization should be contacted through an official and independently verified phone number.

Ethical Considerations:

These scripts were created only for awareness training in a lab environment. They were not used to contacting real individuals or organizations.

Conclusion:

This task demonstrated how social engineering attacks can combine authority, fear, urgency, and pressure to influence a target. Recognizing these psychological techniques can help users identify suspicious calls and respond safely.

OSINT + SE: Build a Target Profile

Objective:

The objective of this task was to understand how publicly available GitHub information can be aggregated into a target profile using Python. The exercise demonstrates how OSINT can be used to identify publicly exposed technical and personal information.

Target Source: Public GitHub profile: “torvalds”

Tools used: Python, requests, json and GitHub public API.

Information collected: The script collected public profile information such as name, company, location, number of public repositories, programming languages found in public repositories, and biography information when available.

```
{
  "name": "Linus Torvalds",
  "company": "Linux Foundation",
  "location": "Portland, OR",
  "public_repos": 12,
  "top_langs": {
    "OpenSCAD": 1,
    "C": 7,
    "Python": 1,
    "C++": 1
  },
  "bio": null
}
```

Attacker Perspective:

From an attacker’s perspective, this information could help build a basic profile of a target. Public repository languages can reveal the technologies a person works with, while the company, location and biography may provide context about their role or interests. This information could potentially be used to make social-engineering messages appear more relevant or believable.

For example, knowing that a person works mainly with Python and Java could allow an attacker to create a message related to those technologies. Similarly, public company or location information could be referenced in a targeted phishing or impersonation attempt.

Defender Perspective:

From a defensive perspective, this exercise shows the importance of reviewing what information is publicly visible on developer platforms. Users and organizations should avoid exposing unnecessary personal or organizational details and should regularly review public profiles and repositories.

Limitations:

The script only uses publicly available GitHub data and does not verify whether every profile field is current or accurate. Missing values appear as “null”, and the programming language counts only reflect the repositories analyzed by the script.

Ethical Considerations:

The task was limited to public information available through the GitHub API. No private repositories, credentials, personal contact details, or unauthorized systems were accessed.

Conclusion:

This exercise demonstrated how separate pieces of public information can be combined into a simple OSINT profile and why organizations should be aware of the information exposed through public developer accounts.

Spear Phishing Email Craft (Lab Only)

Objective:

The objective of this task was to understand how spear phishing uses personalization and social-engineering techniques to make fraudulent emails appear more convincing. The exercise was performed only as an awareness-training simulation in a lab environment.

Tools Used: Python standard library and formatted strings.

Method:

A Python method was created to generate personalized phishing-awareness email templates. Each email used fictional information such as name, email address, company and location. The templates also included urgency and account-related language to demonstrate how spear-phishing messages can pressure users into reacting quickly.

Emails Drafts Created:

Three fictional personalized email drafts were generated. Each draft included a different target name, organization, email address and location. The emails were created only for training and were not sent to real users.

Social Engineering Techniques Observed:

The simulated emails used personalization to make the message appear relevant to the target. They also used fear and urgency by suggesting that an account could be suspended or that immediate action was required. These techniques can make a recipient more likely to trust the message without verifying it first.

SPF / Sender Policy Framework:

SPF allows a domain owner to specify which email servers are authorized to send email on behalf of that domain. Receiving servers can use this information to help detect unauthorized senders.

DKIM / DomainKeys Identified Mail:

DKIM adds a cryptographic signature to outgoing email. The receiving server can verify the signature using a public key published by the sending domain. This helps confirm that the message was authorized by the domain and was not altered during delivery.

DMARC / Domain-based Message Authentication, Reporting and Conformance:

DMARC works together with SPF and DKIM. It allows a domain owner to define how receiving mail systems should handle messages that fail authentication checks. Depending on the policy, messages may be monitored, quarantined or rejected.

Defensive Perspective:

Organizations can reduce the risk of email spoofing by correctly configuring SPF, DKIM and DMARC. Users should also verify suspicious requests through official communication channels instead of trusting urgent links or requests contained in unexpected emails

Ethical Considerations:

The exercise was limited to fictional targets and lab-only awareness examples. No real credentials, real users, or unauthorized systems were involved.

Conclusion:

This task demonstrated how personalization can increase the credibility of spear-phishing emails and why email-authentication controls such as SPF, DKIM, and DMARC are important defensive measures.

Password Attacks & Credential Stuffing:

Objective:

The objective of this task was to understand how repeated password attempts work and how defensive controls such as rate limiting can reduce this risk. The exercise was performed only against a local Flask login server in a controlled lab environment. The internship guide specifically asks for a local brute-force simulation together with a defensive Flask rate limiter.

Tools Used: Python, requests, flask and a local server running on localhost.

Method:

A local Flask login endpoint was created with the username “admin” and the password “letmein”. A Python script then sent POST requests to the login endpoint using a small, predefined password wordlist. The simulator tested passwords one by one until the correct password was found.

Initial Test Result:

```
[-] Failed: password
[-] Failed: admin
[+] Found: admin:letmein
```

This demonstrated how a small dictionary-style password attack can automate repeated login attempts.

Defensive Rate Limiting:

The Flask server was then modified to count failed login attempts. A maximum of three failed attempts was allowed. After the limit was reached, the server returned HTTP status code 429 and blocked further attempts.

The simulator displayed:

```
[-] Failed: password
[-] Failed: admin
[-] Failed: qwerty
[!] Rate limit triggered: Too many failed attempts.
[!] Further login attempts were blocked.
```

Attack Concepts:

Brute Force:

A brute-force attack systematically tries possible passwords until the correct one is found.

Dictionary Attack:

A dictionary attack uses a predefined list of common or likely passwords. The lab script behaves more like a small dictionary attack because it tests a fixed wordlist.

Credential Stuffing:

Credential stuffing involves trying previously exposed username/password combinations on other services. This lab did not use real or leaked credentials.

Defensive Measures:

The internship guide identifies protections such as account lockout, CAPTCHA, MFA, breach monitoring, and rate limiting. In this lab, rate limiting was used to stop repeated failed login attempts after a defined threshold.

Ethical Considerations:

The exercise was performed only on a local Flask server running on the same computer. No real accounts, external websites, or unauthorized systems were targeted.

Conclusion:

This task demonstrated how repeated password attempts can be automated using Python and how rate limiting can help reduce the risk of automated login attacks. The comparison between the unprotected and rate-limited server showed why login systems should include defensive controls against repeated authentication attempts.

USB Drop Attack Simulation:

Objective:

The objective of this task was to simulate a benign USB-drop payload in a controlled environment. The Python script collected basic system information when executed and saved the results to a local text file. The purpose was to demonstrate what a real USB-based payload could try to collect, without performing any harmful action.

Tools Used:

Python standard libraries: “platform”, “socket”, “datetime” and “os”. These libraries were used to collect the current timestamp, hostname, operating system, version, username, and working directory.

Simulation Result:

The script created a file named recon_log.txt containing basic system information.

```
recon_log.txt
1  timestamp: 2026-09-23 11:48:06
2  hostname: DESKTOP-*****
3  os: Windows
4  version: 10.0.*****
5  user: W*****
6  cwd: C:\*****\*****\*****\SQROCK_cybersecurity\sqrock_phase1
```

Sensitive local information such as the hostname, username, operating-system version details, and directory path was anonymized before submission.

USB Drop Risk:

A USB drop attack involves leaving a USB device in a location where someone may find it and connect it to a computer. The internship material explains that malicious USB devices may be used to execute unwanted code and collect information from a system. It also identifies AutoRun abuse as a potential risk on Windows systems.

USB Drop Prevention Policy:

Unknown or untrusted USB devices should never be connected to organizational computers. AutoRun should be disabled to reduce the risk of automatic execution from removable media. Endpoint DLP controls should be used to monitor or restrict the use of unauthorized removable devices. Employees should also receive security-awareness training so they know to report suspicious or found USB devices to IT or security instead of connecting them to their computers. Only approved and trusted removable media

should be used on company systems. These controls align with the defenses listed in the internship guidance: disabling AutoRun, endpoint DLP, and user-awareness training.

Ethical Considerations:

The simulation was performed only on the local computer using a benign script. The script did not install software, modify system settings, transmit collected information, or interact with external systems.

Conclusion:

This task demonstrated how a simple USB-based payload could collect basic host information when executed. It also showed why organizations should control removable media and apply preventive measures such as disabling AutoRun, using endpoint DLP, and training users to avoid connecting unknown USB devices.

Social Media Impersonation and Fake Profile Detection:

Objective:

The objective of this task was to detect potentially fake or suspicious social media profiles using simple behavioral heuristics. The internship guidance identifies indicators such as a very new account, an unusual follower-to-following ratio, lack of a profile picture, very low posting activity, and a default bio. The task is to create a Python fake-profile scorer and analyze anonymized profile samples.

Tools Used: Python standard library

Method:

A Python function was used to assign a fake-profile risk score based on five indicators. The scoring rules were:

- Account age under 30 days: +30
- Following-to-followers ratio greater than 10: +25
- No profile picture: +20
- Fewer than 5 posts: +15
- Default bio: +10

The final score is limited to a maximum of 100%.

Profile Results:

Profile	Age	Followers	Following	No Profile Picture	Posts	Default Bio	
Profile 1	7 days	2	900	Yes	1	Yes	
Profile 2	1200 days	4500	320	No	1	No	
Profile 3	18 days	12	420	Yes	3	Yes	
Profile 4	850 days	300	250	No	95	No	Profile 1 -> Fake Score: 100%
Profile 5	45 days	20	400	No	10	Yes	Profile 2 -> Fake Score: 15%
Profile 6	10 days	150	100	Yes	2	No	Profile 3 -> Fake Score: 100%
Profile 7	1200 days	2500	600	No	400	No	Profile 4 -> Fake Score: 0%
							Profile 5 -> Fake Score: 35%
							Profile 6 -> Fake Score: 65%
							Profile 7 -> Fake Score: 0%

Analysis:

- **Profile 1 (100%):** This profile triggered all five suspicious indicators. The account is only 7 days old, follows far more accounts than it has followers, has no profile

picture, has only one post, and uses a default bio. These factors produced a maximum score of 100%.

- **Profile 2** (15%): This profile is old, has a normal follower/following ratio, includes a profile picture, and does not use a default bio. However, it has only one post, which triggered the low-activity rule and added 15 points.
- **Profile 3** (100%): This profile is 18 days old, has a very high following-to-followers ratio, has no profile picture, has only three posts, and uses a default bio. All scoring conditions were triggered, resulting in a 100% score.
- **Profile 4** (0%): This account is well established, has a balanced follower/following ratio, uses a profile picture, has many posts, and does not use a default bio. None of the suspicious conditions were triggered.
- **Profile 5** (35%): This profile is older than 30 days and has a profile picture and more than five posts. However, it follows 400 accounts while having only 20 followers, which creates a ratio above 10 and adds 25 points. The default bio adds another 10 points, producing a total score of 35%.
- **Profile 6** (65%): This account is only 10 days old, has no profile picture, and has only two posts. These indicators add 30, 20, and 15 points respectively, resulting in a score of 65%. The follower/following ratio and bio did not trigger additional points.
- **Profile 7** (0%): This profile is old, has an established posting history, has a profile picture, and does not use a default bio. Its follower/following ratio also remains below the suspicious threshold. As a result, it received a score of 0%.

Security perspective:

Fake or impersonation profiles can be used to build trust before a social-engineering attack. Attackers may create new accounts, imitate real users, or use generic profiles to interact with potential targets.

However, the score should only be treated as a heuristic risk indicator. A legitimate user may have a new account, few posts, or no profile picture, while a sophisticated fake account may appear well established. Therefore, a high score does not prove that an account is fake, and a low score does not guarantee that it is genuine.

Privacy and Ethical Considerations:

The profiles should be anonymized before submission. Names, ..., contact information, and other identifying details should not appear in the report. Only the behavioral information required for the scoring exercise should be retained.

Conclusion:

This task demonstrated how simple behavioral indicators can be combined into a Python-based scoring system for detecting potentially suspicious social media profiles.

The results showed that recently created accounts with unusual following behavior, limited activity, and incomplete profile information receive higher scores. The exercise also demonstrated the importance of treating automated scores as warning indicators rather than definitive proof of impersonation or fraud.

Baiting and Watering Hole Attack Simulation

Objective:

The objective of this task was to understand the logic behind baiting and watering-hole attacks and to build a Python honeypot link tracker. The tracker was designed to log information about a visit, including the time, IP address, requested path, and browser User-Agent.

Tools Used: Python Standard Libraries: http.server, datetime and json.

Method:

A local HTTP server was created using HTTPServer and BaseHTTPRequestHandler. For safety, the server was bound to 127.0.0.1:8080, which limited the simulation to the local computer. So when the page opened in the browser, the server recorded a log entry containing time, ip, path and agent. The browser then displayed “Thanks for visiting!”

Baiting and Watering Hole Concepts:

Baiting is a social-engineering SE technique that uses something attractive or tempting, such as a USB device or fake download link, to encourage interaction. A watering-hole attack targets a website that a “victim” is likely to visit and uses that trusted location as part of the attack. These techniques can be reduced through technical and user-awareness controls.

Log Analysis:

The honeypot demonstrated how even a simple web request can reveal metadata such as the time of access, source IP, requested resource, and browser User-Agent. In this lab, the only “captured victim” was a simulated local visit from the same computer. No real users or external systems were involved.

Mitigation Measures:

There are several defensive measures against baiting and watering-hole attacks:

- **Web filtering:** blocks access to known malicious or suspicious websites.

- **Script blocking:** limits or prevents untrusted scripts from running in the browser.
- **Patch management:** keeps browsers, operating systems, and applications updated so known vulnerabilities cannot be easily exploited.

User awareness is also important. Users should avoid suspicious download links, verify unexpected content before interacting with it, and report unusual websites or messages to the appropriate security team.

Ethical Considerations:

The test was performed only on the local machine using 127.0.0.1. No deceptive links were sent to real people, and no external systems were targeted.

Conclusion:

This task demonstrated how a simple honeypot can record basic information about web requests and how baiting or watering-hole techniques can rely on user interaction. It also highlighted the importance of web filtering, script blocking, patch management, and user awareness in reducing the risk of these attacks.

Social Engineering Awareness Training Module:

Objective:

The objective of this task was to design and develop an interactive social-engineering awareness quiz using Python. The quiz was created to help users recognize common social-engineering techniques and understand how to respond safely to suspicious situations. The internship guidance emphasizes realistic scenarios, immediate feedback, and repetition as important elements of effective awareness training.

Tools Used: Python standard library and the json module.

Method:

A command-line quiz was created with 12 scenario-based multiple-choice questions. Each question contains three possible answers labeled A, B, and C.

The questions cover topics studied during the previous tasks, including phishing, spear phishing, vishing, smishing, social-media impersonation, fake profiles, password attacks, rate limiting, watering-hole attacks, OSINT exposure, and suspicious account-verification requests.

For each question, the user enters an answer. The program compares the selected answer with the correct answer and immediately informs the user whether the response was correct. If the answer is incorrect, an explanation is displayed.

Score Tracking:

The program keeps track of the number of correct answers throughout the quiz. At the end, the final score is displayed in the following format: "Score: XX/12"

JSON Score Report:

After the quiz is completed, the program creates a file named: "quiz_score.json"

The JSON file stores:

- Final score
- Total number of questions
- Question number
- Question text
- User's selected answer
- Correct answer
- Whether the response was correct
- Explanation for the answer

Awareness Training Approach:

The quiz uses realistic security situations rather than only theoretical definitions. Immediate feedback helps the user understand why an answer is correct or incorrect.

Result:

The quiz successfully ran through all 12 questions, calculated the final score, and generated the "quiz_score.json" file containing the recorded answers and results.

Conclusion:

This task demonstrated how Python can be used to create an interactive cybersecurity-awareness training tool. The quiz helps users practice recognizing common social-engineering situations while providing immediate explanations and keeping a structured record of their performance.

Phishing Email Detection with Machine Learning:

Objective:

The objective of this task was to build a basic Machine Learning classifier capable of distinguishing phishing emails from legitimate emails using a Naive Bayes classifier with text-based features. The model was then evaluated using a confusion matrix, accuracy score and classification report.

Tools and Libraries Used: Python, scikit-learn, CountVectorizer, MultinomialNB, Pipeline, train test split, accuracy score, *confusion_matrix* and *classification_report*.

Method:

The email dataset was labeled using: 1:Phishing, 0: Legitimate

"CountVectorizer" was used to tokenize the email text and convert the words into numerical word-frequency features. These numerical features were then passed to a "MultinomialNB" Naive Bayes classifier.

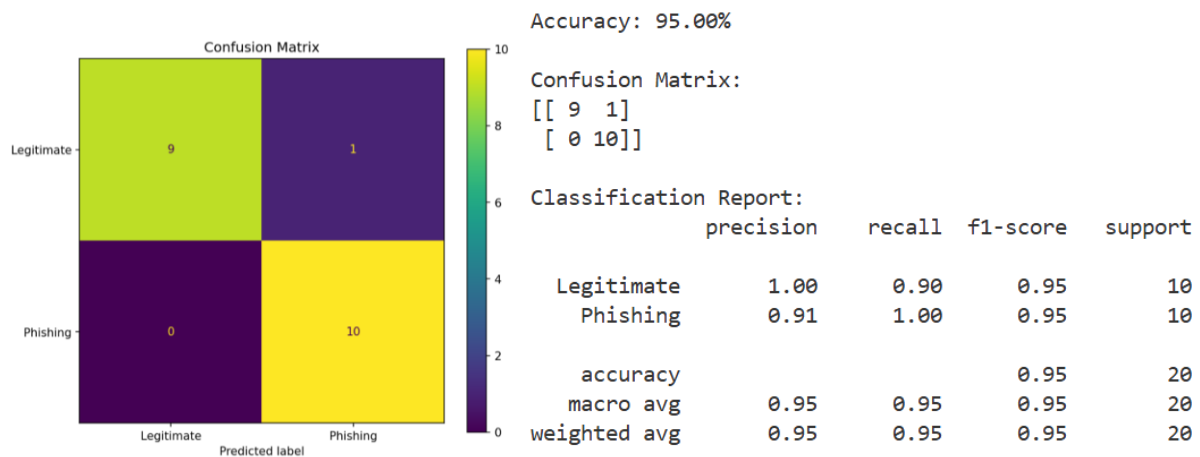
A scikit-learn "Pipeline" combined the vectorization and classification steps so that raw email text could be passed directly to the model.

The dataset was divided into training and testing portions using:

```
test_size=0.35,  
random_state=42,  
stratify=labels
```

The training data was used to teach the classifier, while the testing data was kept separate so that the model could be evaluated on unseen emails.

Model Results:



This means that 9 legitimate emails were correctly classified as legitimate, while 1 legitimate email was incorrectly classified as phishing. All 10 phishing emails were correctly detected, and no phishing email was incorrectly classified as legitimate.

Both classes achieved an F1-score of 0.95, showing a good balance between precision and recall.

Limitations:

Although the model achieved 95% accuracy on this test split, the dataset is still relatively small. The reported performance therefore applies to this particular dataset and train/test split and should not be interpreted as guaranteed performance on real-world email traffic. A larger and more diverse dataset would provide a more reliable evaluation.

Conclusion:

This task demonstrated how Natural Language Processing and machine learning can be used to classify phishing emails. “CountVectorizer” transformed email text into numerical features, while Multinomial Naive Bayes learned patterns associated with phishing and legitimate messages. The final model achieved 95% accuracy and successfully detected all phishing messages in the test set, with only one legitimate message incorrectly flagged as phishing.

SIEM Log Analysis for SE Attack Detection:

Objective:

The objective of this task was to analyze security logs and identify suspicious activity that could be related to social-engineering incidents. The Python script searched the provided sample logs for repeated failed login attempts and suspicious email-rule creation.

Tools Used: Python standard Libraries: re and collection.Counter

Method: The script used regular expressions to search the log data for two types of events:

- “FAILED_LOGIN” events, including the username and IP address
- “EMAIL_RULE_CREATED” events, including the affected username

The Counter class was used to count how many failed login attempts occurred for each user. The brute-force detection rule was configured to generate an alert only when a user had at least three failed login attempts.

Observed Results:

The sample logs contained two failed login attempts for the admin account from the same IP address:

FAILED_LOGIN user=admin ip=45.33.32.156

FAILED_LOGIN user=admin ip=45.33.32.156

Because the threshold was set to three failed attempts, these events did not trigger the brute-force alert.

The logs also contained: EMAIL_RULE_CREATED user=admin rule=forward_all

This event triggered the alert (“Suspicious email rule created by: admin”)

Analysis:

The email-rule creation is suspicious because a forwarding rule can be used to redirect messages from a compromised account. This type of activity may be important during an investigation because it can indicate that an attacker is attempting to maintain access to information after gaining control of an email account.

The failed login attempts are also relevant, even though they did not reach the configured alert threshold. Two failed attempts followed by a successful login from the same IP address may deserve additional investigation when combined with the suspicious email-rule creation.

Defensive Perspective:

A security analyst should review the account activity, confirm whether the email rule was intentionally created by the user, and examine related authentication events. The account, source IP, login times and email configuration changes can provide additional context for determining whether the activity was legitimate or suspicious.

Limitations:

The analysis was performed only on the sample logs provided. The script uses simple pattern matching and fixed thresholds, so it does not consider wider context such as normal user behavior, geographic information, device history or additional SIEM data sources.

Conclusion:

The log parser successfully identified the suspicious email-rule creation in the sample data. The brute-force rule did not trigger because only two failed login attempts were present, which was below the configured threshold of three. This task demonstrated how simple Python-based log analysis can help identify suspicious activity that may require further investigation.

SE Incident Response Plan:

Objective:

The objective of this task was to simulate a simple incident-response workflow for a social-engineering incident. A Python script was used to evaluate an incident based on its type and severity, select appropriate response actions, display them in the terminal, and save the result in a JSON file.

Tools Used: Python standard libraries: datetime and json.

Incident Scenario:

The simulated incident was defined as:

```
"incident": {  
  "type": "phishing",  
  "severity": "HIGH",  
  "user": "riyasqrock.com"
```

Because the severity was classified as HIGH, the script selected several containment actions. Since the incident type was phishing, additional phishing-specific actions were also added.

Response Actions Generated: The script produced the following actions:

```
Actions Taken:  
[x] LOCK user account  
[x] Revoke active sessions  
[x] Notify SOC team  
[x] Preserve mail logs  
[x] Quarantine email  
[x] Block sender domain  
[x] Scan attachments in sandbox
```

These actions represent simulated incident-response steps only. The program does not actually lock accounts, block domains, or modify email systems.

JSON Report:

The script created a file named: ir_report.json

The JSON file stored:

- incident type
- severity
- affected user
- selected response actions
- timestamp of the incident-response simulation

This provides a structured record of the simulated incident.

Incident Response Phases:

The response process can be organized into six phases:

- **Preparation:** establish tools, responsibilities, communication channels, and response procedures.
- **Identification:** confirm that an incident occurred and determine its type and severity.
- **Containment:** limit the impact by protecting the affected account and preserving evidence.
- **Eradication:** remove the cause of the incident, such as malicious emails or blocked sender infrastructure.
- **Recovery:** safely restore normal access and monitor the affected systems.
- **Lessons Learned:** document the incident and identify improvements for future response activities.

Limitations:

The program is a simulation. The listed actions are printed and saved to JSON but are not automatically executed on real systems. Real incident response would also require verification, evidence collection, authorization, and coordination with security teams.

Conclusion:

This task demonstrated how Python can be used to automate the first stage of a social-engineering incident-response workflow. The script evaluated the incident severity and type, generated appropriate containment actions, and created a structured JSON record for documentation.

Final Project: SE Attack Chain Simulator

Objective:

The objective of this project was to build a complete command-line social engineering simulation tool for ethical lab use. The project integrates several previously developed components into one interface, including passive OSINT, public profile analysis, phishing URL scoring, awareness email generation, and incident response simulation.

Project Overview:

The final program was developed as a Python command-line interface named `se_chain.py`. From the main menu, the user can select one of several modules and run it without opening each script separately.

The integrated modules are:

1. Passive OSINT Scanner
2. Public GitHub Profile Analysis
3. Phishing URL Scorer
4. Awareness Email Template
5. Incident Response Workflow
6. Exit

Each function is stored in a separate Python module and imported into the main program.

Module 1: Passive OSINT Scanner

The OSINT module collects publicly available information about an authorized or practice domain. The module uses: WHOIS, DNS resolution, IP geolocation.

The output can include the domain registrar, resolved IP address, and approximate IP location. This module demonstrates how publicly accessible information can be collected during the reconnaissance stage of a social engineering assessment.

Module 2: Public Profile Analysis

The profile module uses public GitHub information to create a simple technical profile. The collected fields include: Name, Company, Location, Number of public repositories, Repository programming languages, Public bio

The purpose of this module is to demonstrate how publicly available information can contribute to a social engineering profile and why limiting unnecessary public information can reduce exposure. The profile-building component uses publicly available GitHub data as part of the social-engineering threat-profile process.

Module 3: Phishing URL Scorer

The phishing detection module analyzes a URL and calculates a simple risk score. The scoring logic checks indicators such as:

- Use of HTTP instead of HTTPS
- Suspicious words such as login, verify, secure, or account
- Large numbers of subdomains
- Raw IP addresses

The final value is limited to a maximum score of 100%.

This module is a heuristic detector, meaning that the score indicates suspicious characteristics but does not prove that a website is malicious. The URL scoring approach is based on observable phishing indicators.

Module 4: Awareness Email Template

The email module generates a fictional phishing-awareness training message using fictional values such as a name, email address, and location. The generated message demonstrates common social engineering techniques such as: Personalization, Urgency, Account warnings, Requests to follow a link.

The program only generates text. It does not send email or contact real users.

The purpose of this component is to show how personalized information can make a social engineering message appear more convincing while keeping the demonstration within an awareness-training environment.

Module 5: Incident Response Workflow

The incident response module simulates a security response after a social engineering alert. The user can provide information such as: Incident type, Severity, Affected fictional user.

For high or critical incidents, the workflow can generate simulated response actions such as: Lock user account, Revoke active sessions, Notify SOC team, Preserve mail logs.

For a phishing incident, additional actions can include: Quarantine email, Block sender domain, Scan attachments in a sandbox.

The results are written to: ir_report.json

These actions are simulated only. The program does not actually lock accounts, modify email systems, or block real domains. The incident-response process follows the phases of Preparation, Identification, Containment, Eradication, Recovery, and Lessons Learned.

Program Structure:

The project was divided into separate files:

- se_chain.py
- osint_module.py
- profile_module.py
- phish_module.py
- template_module.py
- ir_module.py

se_chain.py acts as the main interface and imports functions from the supporting modules. This modular design makes the program easier to understand, test, debug, and extend.

Testing and Demonstration:

The complete program was tested through the command-line interface. During the demonstration:

The OSINT module was tested with a safe practice domain.

The profile module was tested using public GitHub data.

The phishing scorer was tested with synthetic URLs.

The awareness email module used fictional information.

The incident response module generated simulated actions and an ir_report.json file.

A screen recording was created to demonstrate the operation of the integrated tool from the main menu.

Ethical and Safety Considerations:

All activities were kept within an educational and authorized environment.

The project avoided:

Real phishing campaigns

Sending deceptive emails

Credential collection

Scanning unauthorized systems

Using real victims

Executing real containment actions

Public information used during OSINT activities should be handled carefully, and collected information should be anonymized or removed when no longer needed.

Testing should remain limited to lab systems, owned systems, or explicitly authorized environments.

Limitations

The project is a simplified simulation and does not represent a full production security platform.

Some limitations include:

- The phishing URL scorer uses basic heuristic rules and can produce false positives or false negatives.
- Public profile analysis depends on the information available through GitHub.
- The awareness email module generates text only and does not send messages.
- Incident response actions are displayed and recorded but are not executed on real systems.
- The tool does not replace professional SIEM, email-security, threat-intelligence, or incident-response platforms.

Conclusion:

The Social Engineering Attack Chain Simulator combined multiple security-awareness and defensive components into a single Python command-line tool.

The project demonstrated how passive reconnaissance, public profile analysis, phishing detection, awareness training, and incident response can be connected within one controlled simulation.